Semantic XPath 结构化记忆访问方法阅读报告

oioioi-521

(天津大学 NLP 第 8 队)

摘　要:Semantic XPath 结合树路径导航、位置筛选与连续语义评分访问树形对话记忆。本文介绍其树形数据模型、查询语法与带权执行机制，结合单轮、多轮自建任务与 LoCoMo 基准分析其在检索准确性与上下文开销之间的权衡，并探讨层级结构质量与语义评分器对访问效果的影响。

关键词: 结构化记忆；对话智能体；Semantic XPath；语义检索；论文阅读

中图法分类号: TP18

A Reading Report on Semantic XPath for Structured Conversational Memory Access

oioioi-521

(NLP Team 8, Tianjin University)

1　Semantic XPath：树形结构化记忆的查询访问

Semantic XPath[1]聚焦结构化对话记忆的精准访问，结合树路径导航、位置筛选与连续语义评分，从层级记忆树中定位目标节点及子树，为下游回答与状态更新提供上下文。

1.1　研究问题与方法定位

在长期任务导向对话中，用户常围绕同一任务对象（如旅行计划、待办清单）反复查看、修改或回溯。其中一项具体活动不仅包含文本描述，还从属于特定日期、行程和历史版本。直接输入全量历史开销高昂；扁平检索虽能缩减输入，却易召回语义相似但所属日期或版本错误的条目。例如“在会议最密集的一天增加咖啡休息”，需要先按日期聚合评估各天活动，而非仅找一条含“会议”字样的文本。Semantic XPath 据此将结构路径约束与软语义匹配结合，专门负责在树形记忆中锁定访问范围，再交由下游完成回答或节点增删改。

1.2　树形数据模型与查询语法

系统将记忆表示为有根树 $T = (V, E, r)$，非根节点具有唯一父节点，各节点包含类型标签与文本属性，树模式 Σ 由数据自身的节点类型、属性与父子关系自然形成。旅行规划场景采用

$$\text{Itinerary} \rightarrow \text{Version} \rightarrow \text{Day} \rightarrow \text{POI},$$

依次表示行程、版本、日期和活动；上下层体现组成与归属关系，完整保留底层细节而非仅做有损摘要。

查询由步骤序列组成，单步写为 $S = A\,N\,[R]\,[P]$，其中路径轴 A 与节点类型 N 必选，位置选择器 R 与语义谓词 P 可选（表1）。

表 1　Semantic XPath 查询步骤的组成

组成	示例写法	作用
路径 A	/、//	访问直接子节点或全部后代
类型 N	Day、POI、*	按节点类型筛选或匹配任意类型
位置 R	[i]、[-i]、[i:j]	按顺序、倒序或区间选取候选
语义 P	[node~="workshop"]	计算候选与语义条件的相关程度

例如，//Day[3]/POI[node~="workshop"] 先定位第 3 天，再对该日期下的活动计算语义得分；存在多个行程或版本时，可通过前缀路径先限定目标范围。语义条件还支持通过 $1 - P$ 取反，以筛选与指定概念无关的候选节点。

1.3 语义评分与带权执行

节点 u 对语义谓词 P 的满足程度定义为连续相关度 $\mathrm{Rel}(u, P) \in [0, 1]$。除基于当前节点属性的局部评分外，查询还支持嵌套子查询，先对下层节点评分，再用平均（avg）、最小、最大或几何平均聚合下层证据；多个条件可通过乘积、均值及取反组合。

```
//Day[avg(/POI[node~="conference"])]
```

该查询通过聚合每日下属活动的语义得分来评价日期。示例中第 2 天的三个活动得分为 0.603、0.482 和 0.608，均值约 0.565，在候选日期中排名最高，从而被识别为会议最密集的一天。

执行器从根节点及初始权重 1 出发，维护带权集合 $W \subseteq V \times [0, 1]$，按步骤执行路径扩展、类型筛选、位置选择与语义评分更新：

$$w_{\text{new}} = w_{\text{old}} \,\mathrm{Rel}(u, P). \tag{1}$$

遇到聚合条件时递归求值下层子查询，最终按预设阈值或返回数量输出高权重节点及子树。

1.4 范围约束示例

以行程调整为例，假设当前版本第 2 天安排海滨公园，第 5 天安排山地徒步，第 7 天安排植物园和海港游船。面对“找出第 7 天下雨时需调整的户外活动”，纯向量检索易混入其他日期的户外活动；结构化访问则先限定第 7 天子树，再评估户外属性，准确锁定植物园与游船。单层日期约束虽可由带元数据过滤的扁平 RAG 实现，但 Semantic XPath 还能统一处理多级版本嵌套及依赖子节点聚合评分的复杂访问需求。

1.5 实验设置与结果分析

论文在旅行行程、待办清单和家庭配餐三个自建场景（每场景 20 个单轮、5 个多轮对话）以及 LoCoMo 原生会话层级（Memory–Conversation–Session–Turn）上评测。回答模型采用 GPT-5 mini 与 Gemini 3 Flash；相似度评分（XPath Sem）与 Flat RAG 使用 Qwen3-Embedding-8B，蕴含评分（XPath Entail）使用 facebook/bart-large-mnli。自建场景统计任务通过率（需检索正确数据且输出符合约束），LoCoMo 统计模型裁判得分，并将 XPath 查询生成输入计入平均输入 tokens。

表 2 三个自建场景的平均单轮结果

回答模型	方法	通过率（%）	平均 tokens
GPT-5 mini	In-context	70.0	26 216
	Flat RAG	30.0	3 250
	XPath Sem	66.3	6 166
	XPath Entail	83.0	5 691
Gemini 3 Flash	In-context	79.0	25 387
	Flat RAG	37.7	3 361
	XPath Sem	64.3	6 043
	XPath Entail	70.7	5 830

单轮实验（表2）中，GPT-5 mini 配合 XPath Entail 的通过率达 83.0%，较 Flat RAG 提升 53.0 个百分点，且 token 消耗仅为全量上下文的 21.7%；在 Gemini 3 Flash 上，XPath Entail（70.7%）同样大幅优于 Flat RAG（37.7%），但略低于全量上下文（79.0%）。

表 3 三个自建场景的平均多轮结果

回答模型	方法	通过率（%）	平均 tokens
GPT-5 mini	In-context	80.0	69 097
	Flat RAG	26.7	3 327
	XPath Sem	70.0	6 294
	XPath Entail	83.3	5 850
Gemini 3 Flash	In-context	56.7	76 603
	Flat RAG	26.7	3 240
	XPath Sem	76.7	6 092
	XPath Entail	80.0	5 993

在包含多次版本修改的多轮实验（表3）中，XPath Entail 在两种模型下均取得最高通过率（83.3% 与 80.0%），token 用量仅为全量上下文的 8.5% 与 7.8%；同时 Entail 稳定优于 Sem，说明蕴含模型更适合判定条件满足关系。

表 4 LoCoMo 结果（XPath 输入含查询生成）

回答模型	方法	回答得分	平均输入 tokens
GPT-5 mini	In-context	74.55	24 857
	Flat RAG	35.71	1 341
	Semantic XPath	65.19	3 032
Gemini 3 Flash	In-context	87.01	27 499
	Flat RAG	41.49	1 996
	Semantic XPath	73.25	3 870

在 LoCoMo（表4）上，Semantic XPath 仅用全量上下文 12.2%–14.1% 的输入 tokens，便大幅超越 Flat RAG，但仍低于全量上下文（如 GPT-5 mini 多跳题得分为 43.26，低于全量上下文的 58.87）。原文错误分析指出，自由对话缺乏显式业务树，跨分支多跳查询易在节点检索阶段出现漏召回或选错发言节点。

1.6 方法局限与课设启发

Semantic XPath 兼具结构范围约束与评分可解释性，但依赖清晰的层级质量与准确的查询生成，处理跨树关联和复杂多跳仍有局限。与侧重写入期事件绑定的 StructMem 相比，Semantic XPath 侧重读取期范围收缩；课程设计可将二者结合，在检索时先按会话、日期或版本收敛候选范围，再匹配具体事件条目。

References

[1] Liu Y S, Wu R, Gallagher L, et al. Semantic XPath: Structured agentic memory access for conversational AI. In: Durrett G, Jian P, editors, Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), San Diego, California, United States: Association for Computational Linguistics, 2026. 286–296